

Where Does Your Data Live?

Week 1 — Structure, Pointers, Dynamic Memory, and Simple Contracts

Dr. Auqib Hamid Lone

PCC CS-301: Data Structures
GCET, Ganderbal

Week 1

Today's Four Questions

1. How do we organise a program's *logic* before we organise its data?
2. Where does a running C program keep its data?
3. How can we use memory that we request while the program runs?
4. How can we describe a data structure before implementing it?

Plan

Structure the logic. Draw the memory. Use one pointer. Write one simple contract.

Motivation: Correct Is Not Always Fast Enough

A correct program can still be unusable

A loop that scans student records one by one always finds the right student — and with 5 records it is instant.

- ▶ With 10 million records, the *same* correct loop is far too slow.
- ▶ The answer stays right; the *arrangement* of the data is the difference.
- ▶ This course is the catalogue of arrangements, and the judgement to choose between them [2, 1].

Data structures keep a correct program useful as the data grows.

The Course in One Roadmap

1. **Weeks 1–2: Foundations** — structure, memory, pointers, cost.
2. **Weeks 3–5: Linear structures** — arrays, stacks, queues.
3. **Weeks 6–7: Linked structures** — singly/doubly linked lists.
4. **Weeks 8–9: Finding and ordering** — searching, sorting.
5. **Week 10: Files and hashing** — file organization, hash tables.
6. **Weeks 11–12: Connected data** — trees, AVL, graphs.

Each week repeats one shape: *state the contract, then implement it, then analyse what it costs.*

One Running Thread Through the Whole Course

Our running example

A small expression processor reads $(a+b)*c$: check its brackets, convert it, evaluate it, look up its identifiers.

- ▶ A **stack** checks the brackets (Weeks 3–4).
- ▶ A **symbol table** stores the identifiers (Weeks 6, 10).
- ▶ An **expression tree** gives the evaluation (Week 11).

Not twelve unrelated topics — one program learning new tricks each week.

What You Already Know (from prior courses)

- ▶ Variables store values.
- ▶ Arrays store several values in a row.
- ▶ Functions divide a program into pieces.
- ▶ Loops repeat work.
- ▶ C gives you struct for grouping fields.

What this course adds

We ask: *what arrangement of data fits this job?*

Before any structure: how do we organise the program itself?

Section 1

Structured Programming

What Is Structured Programming?

Definition

Structured programming builds a program from exactly three control patterns — sequence, selection, and iteration — instead of unrestricted jumps.

- ▶ **Sequence**: do this, then that.
- ▶ **Selection**: choose one branch (if/else).
- ▶ **Iteration**: repeat while a condition holds (for, while).

Standard treatment; see Horowitz & Sahni Ch. 1 [2].

Worked Example: One Task, Three Patterns

Task: print the larger of two exam marks

```
int larger(int a, int b) {    // sequence: read a, b
    if (a > b)                // selection
        return a;
    return b;
}
```

- ▶ No loop needed here — just sequence and selection.
- ▶ A loop appears the moment we compare more than two marks.

Why Structure the Program Before Structuring the Data

- ▶ A function is a **named, reusable block** — the same idea we will apply to data: name a contract, then implement it.
- ▶ Top-down design: state the goal, break it into smaller functions, then fill each one in.
- ▶ Well-structured code is what makes pointer and memory bugs *findable* — a tangled program hides them.

This course reuses that same discipline for data: state the contract (the ADT) before choosing the implementation.

Before any structure: where does a program even put its data?

Section 2

Memory

A Variable Has a Place

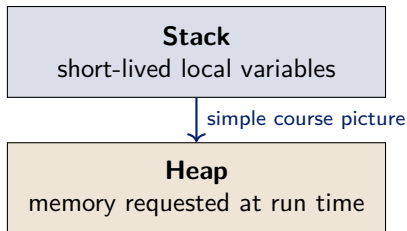
Example

```
int marks = 80; stores the value 80 somewhere in memory.
```

- ▶ The variable has a name: `marks`.
- ▶ It has a value: 80.
- ▶ It also has an address: its place in memory.

An address is like a house number. It tells us where to look.

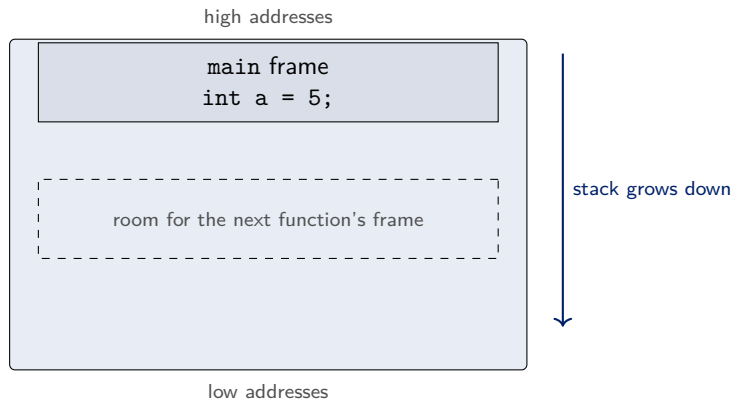
Two Places to Remember



- ▶ Local variables usually live in the stack.
- ▶ Dynamic memory lives in the heap.

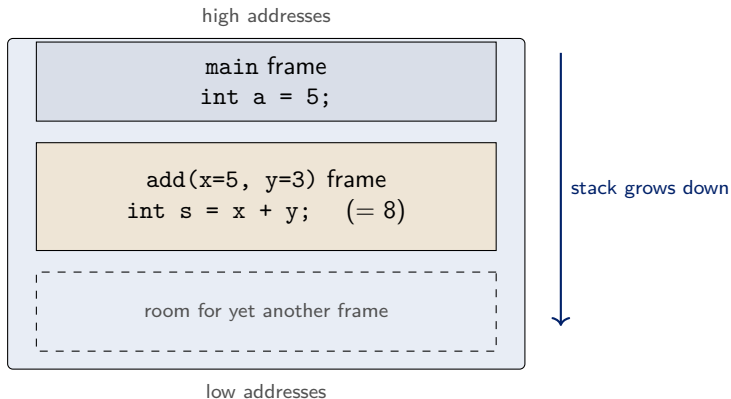
Every later diagram in the course keeps this orientation: stack up, heap down.

How the Stack Grows — Step 1: main starts



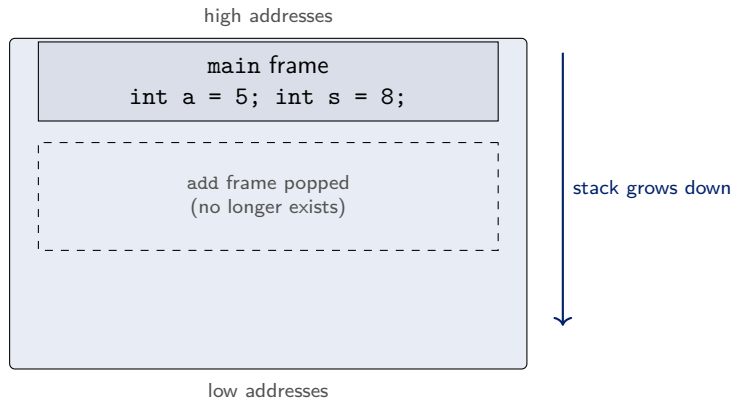
- ▶ Starting `main` creates one frame holding its local variables.
- ▶ Every new function call pushes a new frame just below the current one.

How the Stack Grows — Step 2: add is called



- ▶ The call to `add` pushed a second frame with its parameters and locals.
- ▶ The new frame sits at a **lower** address than `main`'s frame.

How the Stack Grows — Step 3: add returns



- ▶ Returning popped the frame, and its memory is available again.
- ▶ The answer 8 was copied back into `main`'s frame.

The Stack: Automatic Storage

Definition

A local variable is created when its function starts and destroyed when the function returns — the program manages this automatically.

Example

`int score = 80;` inside a function is a local variable: it is gone the moment that function returns.

Why Do We Need the Heap?

A changing problem

The user enters the number of records only after the program starts.

- ▶ We may need space for 3 records today and 300 tomorrow.
- ▶ We may want the records to remain after a helper function returns.
- ▶ A structure whose size is known only at run time cannot be given a fixed amount of storage at compile time [5].

The heap gives us memory whose size and lifetime we request at run time.

Wirth's argument, p.111: if the size is known only at run time, the storage must be allocated at run time too.

Check Your Prediction

Which variable can outlive the function that created it?

```
int score = 80;           or           int *p = malloc(sizeof(int));
```

- ▶ `score` belongs to its function call — gone on return.
- ▶ The heap block remains until released with `free`.

The exact call for this — `malloc` and `sizeof` — is next.

To use the heap, we need a handle on it — a pointer.

Section 3

Pointers

What Is a Pointer?

Definition

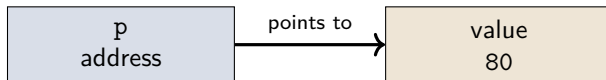
A pointer is a variable that stores an address [5].

- ▶ An ordinary variable stores a value such as 80.
- ▶ A pointer stores a place such as “where the score lives”.

One declaration

`int *p;` means: `p` can point to an `int`.

A Pointer Picture



- ▶ p does not contain the score itself.
- ▶ p tells us where the score can be found.

A Pointer Can Reach a Node

A two-field node

```
struct node {  
    int data;  
    struct node *next;  
};
```

- ▶ data stores the value.
- ▶ next stores the address of another node, or NULL.

This is the building block for a linked list in a later week. (The list handle will be called head; here the node is just p.)

The Safe Empty Pointer

Definition

NULL means a pointer is not pointing to a valid object.

Example

`struct node *p = NULL;` means “this pointer reaches nothing yet”.

- ▶ Check a pointer against NULL before using what it points to.
- ▶ Never read through an empty pointer.

Now let us actually request a block of heap memory and use it.

Section 4

One Heap Block

Requesting Memory: malloc

Definition

`malloc(n)` asks the heap for `n` bytes and returns their address, or `NULL` if the request fails.

```
int *p = malloc(sizeof(int));  
if (p == NULL) return 1;  
*p = 80;
```

Always check the result against `NULL` before using it.

sizeof: the Unit of Every Request

Definition

`sizeof(T)` is the compile-time size, in bytes, of type `T`.

```
sizeof(int)           // typically 4  
sizeof(struct node)   // data field + next field
```

- ▶ `malloc` never knows how big an `int` is on its own.
- ▶ `sizeof` is what tells it — every allocation request uses it.

Zero-Initialized Memory: `calloc`

Definition

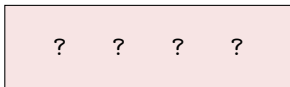
`calloc(n, size)` allocates space for `n` items of `size` bytes each, and sets every byte to 0.

```
int *counts = calloc(5, sizeof(int));  
if (counts == NULL) return 1;
```

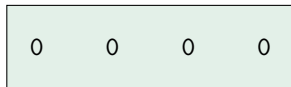
`calloc` zero-initializes; **`malloc`** does not.

Worked Example: malloc vs. calloc

`malloc(4*sizeof(int))`



`calloc(4, sizeof(int))`



- ▶ `malloc` hands back four **unspecified** bytes — reading them before writing is undefined behaviour.
- ▶ `calloc` hands back four bytes already **set to zero**.

Resizing a Block: `realloc`

Definition

`realloc(p, newsize)` resizes the block `p` points to, preserving its existing content up to the smaller of the old and new sizes. The block **may move** — always keep the returned pointer.

```
int *bigger = realloc(p, 10 * sizeof(int));  
if (bigger == NULL) return 1; // p is still valid here  
p = bigger;
```

Worked Example: Growing an Array with `realloc`

Trace

Start with `p = malloc(3 * sizeof(int))` holding 10 20 30.

1. `p = realloc(p, 5 * sizeof(int));`
2. The first three ints (10 20 30) are copied to the new block.
3. The two new slots are **unspecified** — `realloc` does not zero new space, only `calloc` zeros space.

If `realloc` returns `NULL`, the original block at `p` is untouched and still valid — never overwrite `p` with the raw result before checking it.

Allocate a Complete Node

A safer engineering pattern

```
struct node *p = malloc(sizeof(struct node));  
if (p == NULL) return 1;  
p->data = 42;  
p->next = NULL;
```

- ▶ `sizeof(struct node)` asks for exactly one node's bytes.
- ▶ `p->data` means: the data field of the node `p` points at.
- ▶ This node has a clear end: `p->next == NULL`.

Audit the Node Before You Ship It

Review checklist

A small program is not finished when it compiles.

1. Was the allocation checked against NULL?
2. Was every field initialised before the node was used?
3. Is there exactly one clear owner responsible for `free(p)`?
4. Can the program reach the node later, or has its pointer been lost?

Trace It Before Running It

After these lines, what is true?

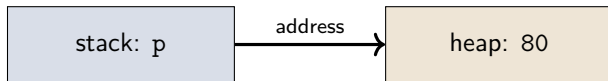
```
int *p = malloc(sizeof(int)); *p = 80;
```

1. p contains an address.
2. The heap block at that address contains 80.
3. The name p itself is not the heap block.

Challenge

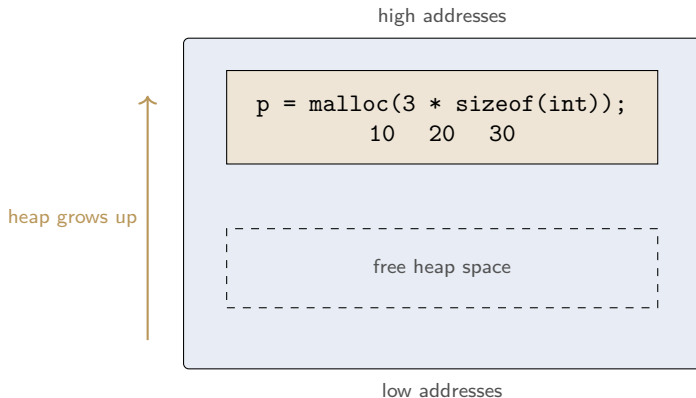
Which line would fail if `p == NULL`?

What Happened?



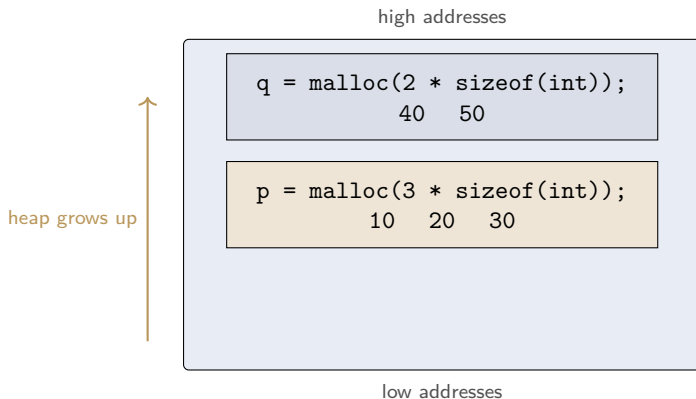
1. C found a free heap block.
2. The address was stored in p.
3. We used *p to write the value 80 there.

How the Heap Grows — Step 1: a block of three ints



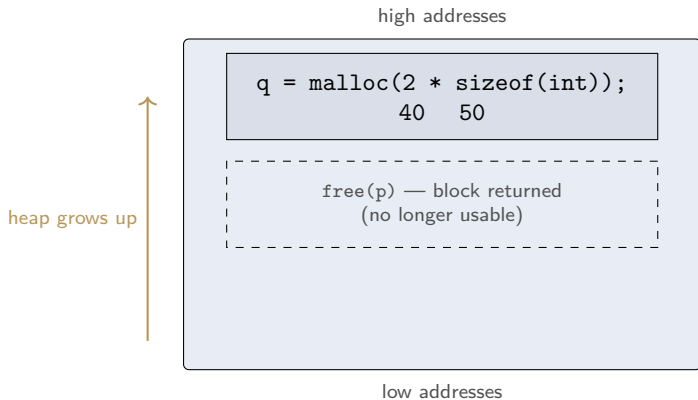
- ▶ One `malloc` with room for three integers **grew the used heap** upward.
- ▶ The block survives after the allocating function returns.

How the Heap Grows — Step 2: a second block



- ▶ The second request placed `q`'s block at a **higher** address.
- ▶ Blocks are independent; each keeps its own data.

How the Heap Grows — Step 3: free returns a block



- ▶ `free(p)` gives `p`'s block back to the system.
- ▶ `q`'s block is untouched, and the used heap can shrink again.

Returning the Memory: `free`

Definition

`free(p)` returns the heap block `p` points to back to the system. Every `malloc/calloc/realloc` needs a matching `free`.

```
free(p);  
p = NULL;
```

Request memory, use it, return it.

Two Beginner Mistakes

- ▶ **Memory leak**: forget to call `free`; the block stays reserved forever.
- ▶ **Dangling pointer**: use a pointer after its block was freed.

Simple habit

Keep track of who requested the memory and who will return it.

Memory and pointers are the machinery. The contract is the map.

Section 5

Simple Contracts

What Is an ADT?

Definition

An ADT describes *what* we can do with data, without saying *how* it is stored — “a set of values and a set of operations on those values” (Sedgewick, p.64) [4].

Karumanchi (p.18, PDF) frames the same idea as a declaration of data plus a declaration of operations [3].

Worked Example: A Very Small Bag

The contract

A bag of integers supports three operations:

1. `insert(x)` — put `x` into the bag.
2. `member(x)` — answer whether `x` is present.
3. `size()` — report how many items are present.

The contract does not yet say whether the bag uses an array or a list. A bag is one of Sedgewick's three collection types (p.120) [4].

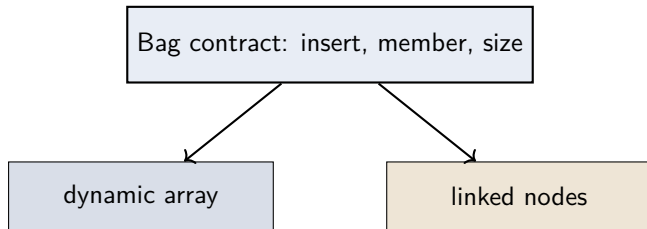
This Course Is a Tour of ADTs

The pattern you will see every week

Linked Lists, Stacks, Queues, Priority Queues, Binary Trees, Hash Tables, and Graphs are all commonly used ADTs [3].

Watch for this shape every week: “What is a Stack?” then “Stack ADT” then “Implementation.”

One Contract, Two Possible Implementations



- ▶ Same operations, different internal arrangement.
- ▶ The caller's code does not change when the bottom box does.

Contract or Implementation?

Classify each statement

Is it part of the contract, or part of one implementation?

1. “`member(7)` answers yes or no.”
2. “The values are stored in an array.”
3. “`size()` reports the number of values.”

The first and third describe what users can ask (contract). The second describes how (implementation).

Week 1 Summary

- ▶ **Structure:** sequence, selection, iteration build every program.
- ▶ **Memory:** the stack holds short-lived locals; the heap holds memory requested while the program runs.
- ▶ **Pointers:** a pointer stores an address; NULL means “points to nothing.”
- ▶ **Dynamic memory:** malloc/calloc/realloc request, free returns.
- ▶ **ADTs:** state operations first (a contract); a data structure is the implementation that honours it.

Next week

We learn how to talk about the amount of work an operation needs — $T(n)$, $O/\Omega/\Theta$, and best/worst/average case.

References I



Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman.

Data Structures and Algorithms.

Addison-Wesley, 1st edition, 1983.



Ellis Horowitz and Sartaj Sahni.

Fundamentals of Data Structures.

Galgotia Booksource, 2nd edition, 2008.



Narasimha Karumanchi.

Data Structures And Algorithms Made Easy.

CareerMonk Publications, 2017.

This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4th edition, 2011.

References II



Niklaus Wirth.

Algorithms and Data Structures.

Author (freely distributed), 2004.

Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.